Agent Modeling Guide

Miles Parker

Copyright 2010 Metascape, LLC

1. Introduction 1
1.1. Agent-Based Modeling 1
1.2. Agent Modeling Framework 1
1.3. Other Uses 2
2. Structure 3
2.1. Design 3
2.2. Types 3
2.2.1. General 3
2.2.2. Agents 4
2.2.3. Attributes 4
2.2.4. Spaces 6
2.3. Reference 8
2.3.1. Diagrams 8
3. Actions 11
3.1. Background 11
3.2. Design 11
3.2.1. Types 11
3.2.2. Flow 14
3.2.3. Boundaries and Query Refinement 18
3.2.4. Weaving 21
3.3. Components 24
3.3.1. Root Actions 25
3.3.2. Selection 26
3.3.3. Query 26
3.3.4. Logic 26
3.4. Example 27
3.5. Reference 28
3.5.1. Diagrams 28
4. Functions 30
4.1. Reference 30
4.1.1. Diagrams 30

Chapter 1. Introduction

In this Modeler's Guide we provide an in-depth understanding of what goes into an Agent Model and how you can use the Agent Modeling Framework tools to design one. Fundamentally, an agent-based model, or "ABM", is composed of five pieces: Agents and Context Agents, Attributes, Spaces, and Actions. The first three refer to structural components, whereas Actions define behavior. Agent models also have styles, which are a special kind of Action used to determine how to portray an agent in a visualization. Finally Actions make use of Functions. We'll describe of these components in a seperate section.

But first, we'll give a quick overview at what agent-based modeling is is and how the Agent Modeling Framework can help you to develop models for ABM or other problem domains.

1.1. Agent-Based Modeling

The primary focus of the Agent Modeling Platform tools is "Agent-Based Modeling" (ABM). ABM is an innovative technique used to explore complex phenomenon in many domains, including economics, social sciences, bio-medicine, ecology and business operations. ABMs share characteristics with object models, but are:

Spatial	Models have explicit environment(s) in which agents interact. (An environment need not be a physical landscape; other examples of spatial relationships include social networks or positions within a logic system.)
Temporal	Models change over discrete units of time.
Autonomous	Agent behaviors are activated independently from other object requests.
Heterogenous	Agents may share behavior definitions but have apparent and distinct states and behaviors.
Collective	Models contain large communities of agents which exhibit collaborative and competitive behaviors.
Emergent	Agents have collective macro-behaviors that are non-obvious from agent micro-specifications.

Existing scientific models are very good at representing relatively simple systems, but generally speaking aren't very good at representing complex systems. The world is full of complex systems, and our misunderstanding of these systems has prevented us from addressing many of the key challenges facing the world, including the global financial crisis and climate change -- in fact once could argue that our misunderstanding of these systems has strongly contributed to these crises.

Agent-Based Models (ABMs) seek to represent important real-world dynamics by designing communities of software agents that mimic real entities. Rather than make simplifying assumptions about such systems and then representing them in equation form or as off the shelf algorithmic constructs, the ABM researcher aims to identify key agent state, interaction spaces, and behaviors. Agents are then "let loose" on our computers and we explore what happens next. The computational horsepower exists today to simulate large numbers (e.g. >>10) of interacting, adaptive and autonomous agents but often desktop computers are all we need to explore significant domains. ABMs have been designed to represent all kinds of important natural systems, at scales reaching from cellular mechanics to international trade and are being used to solve truly hard problems in government, business, and academia. ABMs are not a solution to every problem, but they can help us to appreciate and gain unique insight into many systems, and often they can help us to come up with better practical decisions than we might using classic approaches.

1.2. Agent Modeling Framework

The Eclipse Platform provides many unique features that make it ideal for an ABM platform. AMF provides easy to use and powerful tools and techniques for desiging Agent-Based Models, including a common representation, editors, generators and development environment.

The Agent Modeling Framework (AMF) provides high level representations for commmon ABM constructs, and introduces novel ways of representing agents and their behaviors. As detailed in other documentation sections, the Agent Modeling Framework and related tools have been designed to allow researchers to explore complex models in an intuitive way. One of our major design goals has been to

create tools that non-programmers can use to create sophisiticated models. It has been our experience that using Model-Driven Software Development (MDSD) techniques increase productivity for all developers regardless of skill level. For those familiar with Model-Driven Developemnt tools, AMF is analogous to EMF but is targeted toward the design and execution of models composed of agents. The AMF meta-model (currently "MetaABM", but in the process of evolving to a new compatible "Acore" specification) is defined in Ecore but provides a more direct and high-level ABM representation of agents, including spatial, behavioral and functional features sufficient to generate complete executable models for the target platforms.

Models designed in AMF are transparantly converted to Java code for leading Agent-Based Modeling tools, such as the Escape tools which are included in AMP and allow direct execution of models within the AMP environment, and Repast Simphony, another popular Java based ABM tool. These tools create Java code that can then be compiled, executed and event modified in these environments just as with any other Java program. AMF's generative capability is designed to be pluggable and modular so that other developers can create AMF generators for their own tools. In fact, targets can be designed that have no inherent dependencies on Eclipse or even on a traditional platform.

1.3. Other Uses

Many kinds of objects share characteristics of ABM agents -- after all, software agents are used in many other contexts. The Agent Modeling Framework meta-modeling support is potentially suitable (or extendible) for a number of approaches outside of ABM; for example business rules, object interactions, systems dynamics models and traditional discrete event models. Similarly, AMP execution and graphic support can be used for modeling natural systems but could also be used to manage other software agents -- for example independent reasoning tasks or dynamic visualization support.

We hope you enjoy using the tool, and we look forward to your comments and feedback and most of all participation!

Chapter 2. Structure

2.1. Design

The basic structure of an agent-based model can be quite simple. While there are many subtle complexities -- beyond the scope of this manual -- we can construct most models following some straightforward and elegant design principles. And in fact, one of the main goals of the Agent Modeling Framework is to provide a consistent framework that can support using those principles to support the creation of models that can be eaisily understood, shared, and that can be used interchangably as components in other models.

Unlike the approach of a traditional Object Oriented environment, the dominant organizing principal for agents within AMF follows a compositional hierarchical model, not an inheritance model. (Inheritance-like behavior will be supported in forthcoming versions of Acore, but in a more sophisticated, flexible and dynamic way than is supported by traditional programming languages such as Java.) Contexts -- also referred to as "Swarms" or "Scapes", are simply Agents that are capable of containing other agents. With this basic construct -- known as a Composition in design pattern language -- agents are able to contain other agents.

2.2. Types

2.2.1. General

Everything represented in an Acore model needs to be referred to in some way. Just as software classes have names, Acore provides a way to label and describe every entity, but in a richer and more maintainable way. All entities in Acore -- including Actions and Functions which we descibe in the next two sections -- have a number of shared values.

2.2.1.1. Named Entities

Label

A reasonably short, human readable name for the agent. For example, "Timber Wolf", "Forex Trader" or "Movement Probability". These should be defined so that they fit in well with auto-generated documentation.

ID

An ID is an identifier that can be used to represent the object in a software program. This means that it must follow certain rules such as no-spaces or non alpha-numeric characters. The editing tools will help make sure that this value is legal. Note that when you enter a label, a legal ID is automatically created for you! Usually you won't need to change this, but you might if for example you want the value to match up with some database or other external representation. So reasonable values here might be "timberWolf" or perhaps "MVMNT_PRB" if say you're trying to match up to some old statistics records you have. (Note that currently IDs by default use "camel case", i.e. "thisIsAnExampleOfCamelCase" to match with Java variable naming conventions, but this is likely to change.)

And most entities also define:

Description

A complete textual description of the object. Don't overlook this -- it is the most important part of your model. The description will show up in your auto-generated html documentation, software documentation and even in your running model. You should include enough information that model users will understand what the entity is for and what it does without referring elsewhere. This is also where any attributions and references should go. You can put html tags in here -- such as href's to external papers, but keep those to a minimum as they won't be rendered as html in all contexts.

Plural Label

The plural representation of an entity. This can be a surprisingly useful thing to have in generated documentation and software code, so it's worth maintaining. The editor will automatically add an "s"

at the end of the label you've entered above, but you change it to whatever is appropriate. For example "Person", or "Timber Wolves".

2.2.2. Agents

2.2.2.1. Simple Agents

An Agent is simply a software object that has autonomous behavior and (generally speaking) exists within some set of spaces. By autonomous, we mean that agents make the choice about when and how to execute a behavior, as opposed to have that controlled from "above", so to speak. Like any software objects, agents have attributes (fields) and actions (methods) associated with them.

Attributes

As described int eh attribute sections above.

Actions=

Described in the "Actions" section.

Styles

Special actions that are used to define how to draw an agent grpahically as also described in the "Actions" section and detailed in the "Functions" section.

2.2.2.2. Context Agents (Contexts)

As detailed above, agents also form the basic structural component of an agent-based model. To get an idea for how this works, have a look at the "EpidemicRegional.metaabm" model. Note that in future releases, we will probably refer to Contexts as "Scapes".

Agents

The agents that are contained within this context. For example, a context representing a city might contain an Individual Agent for defining all individuals in the model, and a Vehicle agent defining all vehicles. Note that when we refer to an agent in this context, we mean a general type or class of agent, not the agent itself.

Spaces

The set of all spaces contained or subsumed by the agent. For example, a context representing a city might contain a geographical space and a transporation network space.

2.2.3. Attributes

Agents need some way to represent their internal state. For example, an agent in an economic model might have a wealth attribute, and an agent in an ecology model might have a quantity of food and a particular vision range. These states are represented as attributes just as all software objects do. In an agent model, we keep richer information about these attributes and generally represent them at a higher level. For example, rather than specify that a real value is a "double", a "float" or a "big number", we represent them as "Reals" so that they can be implemented and tested in different environments. This might allow us for instance to ensure that a model's behavior is not dependent on a particular machine implementation of floating point arithmetic. Also, note that attributes only represent so-called "primitive" values -- that is, actual measurable features of a particular agent but not an agent's relationship to other agents or objects. See the discussion about networks for more on this key topic.

Here are the basic types of attributes available in Acore models:

2.2.3.1. Basic Attributes

Attributes are single values that a given agent contains. For example, an agent might have an "Age" attribute. In this section we go over the values you can set for the attrributes. For those with a tecnical

bent, note that we are technically describing the meta-attributes for the meta-class "SAttribute". But it is far too confusing to refer to the attributes of attributes! So we'll just refer to the attributes that any of our model components as "values".

Type

These can be anyone of the following:

Boolean	A value that is simply true or false. Note that unless this value really is a simple binary value, you should consider using state instead. (See details below.) For example, rather than representing gender as a 'Female' boolean value, define a 'Gender' state with values 'Male' and "Female'. Generated artifacts and documentation will be much clearer, and you'll be able to easily modify the model later if for example you discover that there are more than two potential gender categories that are relevant to your model.
Integer	A discrete whole number value, such as "100", "-1", "10029920". It's generally a good idea to represent any value that can never have a decimal value as an integer.
Real	A continuous number value. While these are typically represented in software as floating point numbers, they can conceivably represent numbers at any arbitrary precision and in any scheme. Note that while technically speaking we should be representing irrational numbers, this is not currently supported for default values and users should simply use the closest decimal approximation.
Symbol	A string representing some state. More precisely, a computationally arbitrary value with contextual meaning. This could be any kind of identifier. For example, you might use it to store some kind of input coding from data that is then converted into an object state. Or it could simply be an agent's name. But theoretically (though this is not currently supported) one could imagine a symbol using an idiogram, an icon or even a sound that identifies or represents the agent in question.

(Undefined and Numeric types should not be used within a well-defined model.)

Default Value

The value that should be assigned to the attribute at the beginning of any model run. The attribute may of course be assigned a different value in an Initialize rule, but its a good idea to specify one here. It's OK to leave it blank, in which case a sensible 'empty' value will be assigned, i.e. false for Boolean, 0 for Integers and Reals, and an empty string for Symbol.

Gather Data

Here you can specify whether executing models should collect aggregate values for the data. For example, if you select this value as true for a 'Wealth' attribute, Escape will automatically keep track of minimum, maximum, average, sum and optionally standard deviation and variance across all agents for each model execution period. All of these statistics will then be selectable with a mouse click to appear in your model charts at runtime.

Immutable

This value indicates whether you expect the model value to change. If you know that it won't or shouldn't, this value should be true.

Units

Specifies what the attribute is actually measuring. For example, if you're defining an attribute for "Wealth" in the context of a model of the world economy, you might specify "USD". If you're defining "Age", you might specify "Years", for "Mass" "Kg". Like description, this value is often overlooked, but can be critically important to allowing yourself and others to understand and correctly calibrate a model. Note that this will also allow you to simplify variable names -- instead of using "Age in Years", you can simply specify "Age" and the appropriate unit. It may be obvious to you that your model is concerned with age in years, but a user who needs to specify a different granularity will be grateful for more clarity.

2.2.3.2. Arrays

Arrays are simply attributes with zero or more entries. For example, you might have an array of three Real numbers representing the Red Green and Blue color components for an object. Note that if you find yourself defining very complex sets of arrays, its likely that what you really want to define is a new agent with attributes for each array. In addition to whatis defined above, arrays specify:

Size

The number of values that the array attribute will contain.

2.2.3.3. States

States represent any agent quality that may take on one of a number of well defined values. For example, an "Ice Cream Consumer" Agent might contain a state of "Ice Cream Preference" with options of "Chocolate", "Vanilla" and "Ginger".

State Options

Create new options for states by adding them to the state node. States are simple described items. Don't forget to provide a description! States also have

Default Option

unlike for regular attributes, this option is not optional! Simply pick a state option to be assigned that the agent will take if no other option has been assigned.

2.2.4. Spaces

All contexts can contain spaces. Spaces provide an environment in which agents can have a physical or notional location and upon which they can interact with one another and their environment. Agents can exist in more than one space at a time, and a given agent need not exist in every space. (Note that this is different from the Repast Simphony notion of projections, though the two representational approaches are generally compatible.) Agents need not represent explicit, "real" spatial structures such as a landscape, though this is of course the most common use. They can also represent relational and state information, such as a belief space or a social network. There are four kinds of space represented:

2.2.4.1. Space (Continuous)

In the modeling tools, we simply refer to this as a "Space" as it is represents the general concept of a space. A space is simply something that contains objects with certain locations and extents and has a certain number of dimensions. The space is continuous, in the sense that objects can be placed anywhere with arbitrary position. Spaces hold attributes, which simply define their dimensions-- see below.

Border Rule

A value representing what happens to an agent when that agent is asked to move beyond its extent.

Periodic	When encountering an edge, the agent will treat the space as wrapping around to the other side of the space. For example, if the agent at location 1,2 (0-based) within a Moore space (see grid discussion below) of size 10,10 is asked to find some other agent within distance 3, the agent look in the square defined between 8,9 and 4,5. An agent asked to move beyond the confines of the space will simply stop when it reaches the edge. You can imagine this as taking a piece of graph paper and connecting the opposite edges. You can't actually do that with paper, but if you could you would have a toroidal (donut) shape in three dimensions defining the shape in two.
APeriodic	When encountering an edge, the agent treats it as the edge of the space. For example, if the agent at location 1,2 is asked to find some other agent within distance 3, the agent look between 0,0 and 4,5. An agent asked to move beyond the confines of the space will simply stop when it reaches the edge.

The "Strict" and "Bouncy" values are obsolete and should not be used.

Dimensionality

The number of dimensions that the space has. After selecting a dimensionality, attributes will be added to represent each dimension. For example, if you enter 3 here, you will have an attrbiute for X, Y, and Z. Be sure to enter default values here, as they will be used to specify the actual size of the space.

2.2.4.2. Grid

A grid is technically a regular lattice structure. Currently, only rectilinear strcutures are supported, i.e. a One-dimensional vector, a two-dimensional grid, a three-dimensional cube and so on. (Though none of the current target platfroms support n-d spaces yet.)

Like continuous spaces, a grid has a border rule and dimensionality. A grid has a couple of other important values:

Multi-Occupant

Does the grid allow more than one agent to occupy it at a time? This value can usually be ignored and will likely be deprecated in future releases.

Neighborhood

This value determines what constitutes a region within a particular distance from the agent. The value for this is often ciritical in obtaining particular behavior from a model, and shouldn't be overlooked. There are three possible values:

Euclidian	The distance between any two cells is taken to be the "real" distance. For example, if an agent was within a chess board, and we wanted to find all agents within distance three of it, we could determine that by taking a string of length 3, tacking it to the center of the source square, and including all cells whose centers we can reach with the other string end. Note that although Euclidian space may seem the most reasonable neighborhood configuration to choose, this really isn't the case.

	Euclidian space is continuous whereas grid space is discrete, and mapping the two to each other can create unexpected issues. Still, this is a good choice for models representing notionally real spaces.
Moore	Here, the distance between any two cells is defined by the number of edge or corner adjacent cells crossed to get between them. To continue the chess board analaogy, this is the set of moves that a king can make. Note that this does not map well to real space at all, as a cell at distance 1 in a moore space is at distance sqrt(2) in "real" space.
Von-Neumann	Here, the distance between any two cells is defined by the number of edge adjacent cells crossed to get between them. This is the set of moves that a rook might make on a chess board -- if a rook could only move one square at a time.

2.2.4.3. Network

A network represents a set of relationships between agents, in a graph structure. The concept is pretty simple, but note that a network is actually a critical part of many AMF models. This is because we use networks to represent any kind of references between agents. You may have noticed in the discussion about attributes that there is no generic "object" type for an attribute. In fact, attributes only contain primitive values. So we will use networks to do the kinds of things that we would often use object references for in a traditional Object-Oriented model. For example, if an agent is a member of a family, rather than have that agent have a member attribute of "family" with a reference to the "Family" object, the modeler would create a "Family Members" network and connect agent's to their families as appropriate. This implies that network members can contain members of many different types.

A network has only one value to specify:

Directed

This indicates whether connections between agents are one-way or two-way. If this value is false, then if a connection is made between an agent A and an agent B, and agent B searches within distance 1, agent B will find agent A. If this value is true, agent A can find agent B, but agent B can not find agent A. (Unless of course some other path leads B to A.)

2.2.4.4. Geography

A geography represents a physical landscape. Here we assume that that landscape is going to be defined by an external data source and representational scheme -- typically a Geographical Information System. We'll describe how to work with GIS in more detail when we discuss builder actions.

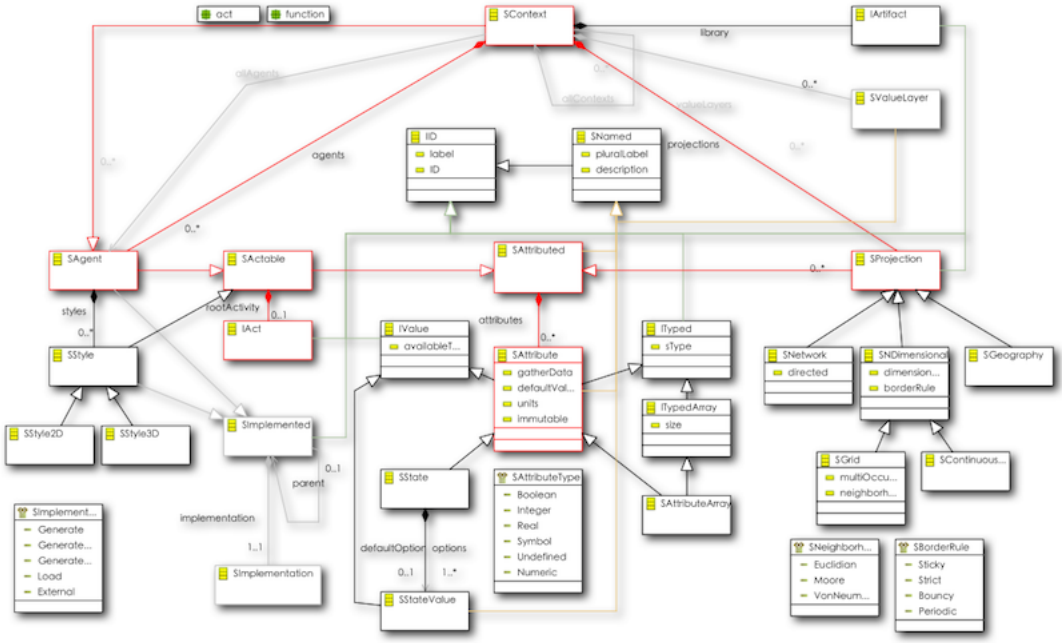
2.3. Reference

2.3.1. Diagrams

For readers familiar with UML and meta-modeling, the following diagrams give more detail on the structural design.

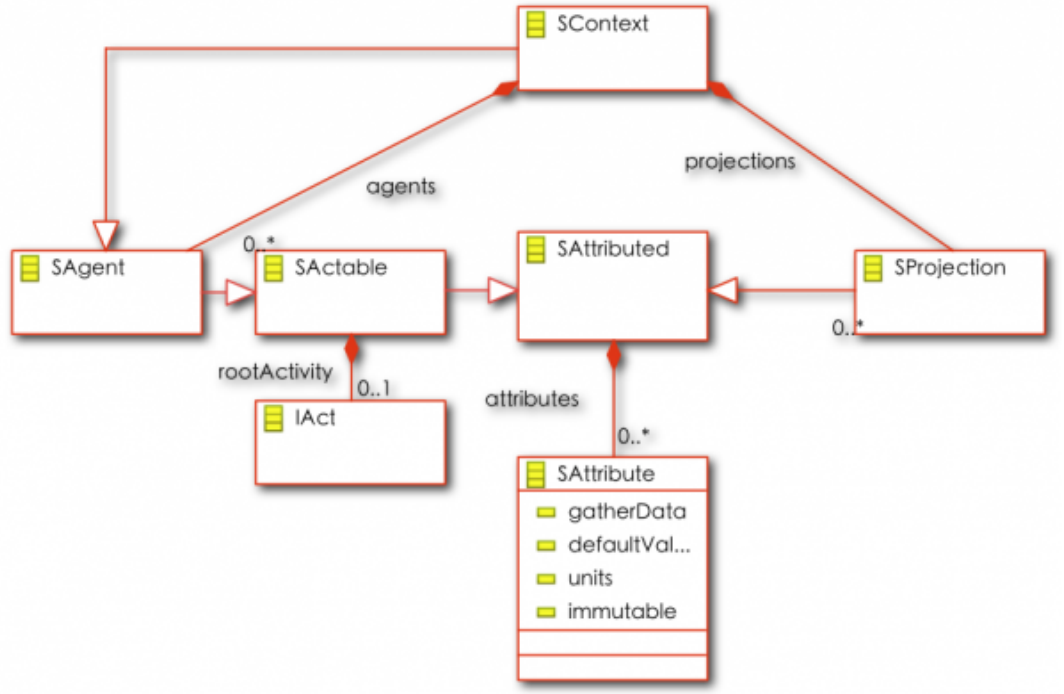
2.3.1.1. Meta-Classes

Our first diagram depicts the core structural design of the model.



There seems to be a lot going on here, but the basic components are pretty straightforward as we can see in the next diagram.

2.3.1.2. Key Collaborations



Core interactions are in Red. The meta-model structure is essentially a Composite pattern.

2.3.1.3. Details

1. Every model has at its root a Context (Scape). Contexts are Agents that are capable of containing other Agents (including other contexts, naturally).
2. (Meta-level) Contexts contain (meta-level) Agents at the model (design-time) level. At runtime, (model-level) Context instances defined in a (meta-level) SContext will contain (model-level) agent instances of the defined (meta-level) SAgent. This sounds more complicated than it is, so let's look at a simple example. Suppose we create a new Context, and give it a label of "Wiki Example". Within that Context, we create an Agent and give it a label of "Individual" and another Agent with the label

"Block". At runtime when we are actually executing the model we will have one WikiExample model instance which contains a number of Individuals.

3. Agents contain Attributes, such as Vision and Age. Context attributes often represent input parameters for contained agents. For example, our Wiki Agent Context might contain an Individual Count as well as a Minimum Age and Maximum Age.
4. Contexts can contain Projections (Spaces), which represent some kind of spatial or structural interaction space for the agents; either a grid, a continuous (euclidian) space, or a network (graph) or geographic space of some kind. For example, we might want to have a City that contains Blocks and that an Individual can move around within.
5. Agents are Actables and thus can contain any number of behaviors called "Actions", described in detail in the next section. Actions can describe individual behavior, and at the Context (Scape) level can define how member Agents and Projections are created.
6. Styles provide a mechanism for defining generic visualization behavior for Agents and so are also Actables. For example, an Agent might have an Action that says effectively "draw a red circle shaded red for the wealth of the agent".

Chapter 3. Actions

Actions are the most important part of an agent model. While Agents, Attributes and Spaces define what we're modeling, it is Actions that give the models life.

3.1. Background

Actions allow the definition of behavior for agents at a very high level. You can think of actions as being analogous to methods in a traditional object-oriented model but that analogy only goes so far. In the same way that methods are defined as part of objects, actions belong to particular agents. (Though even more expressive ways of defining actions are contemplated in future releases.) In the next section we go into detail about what Actions are and how they can be used to define all agent behavior. They are also conceptually more challenging as unlike with structure they have no direct analogies to past agent representations.

An action provides simple, well-defined details that a model can use to determine what steps to take in model execution. That definition seems general enough to be almost useless, but its important to understand that an action is not equivalent to an instruction, a method or a query. In fact, actions can have aspects of all of these. But an action is not in itself an instruction specifying exactly *how* the modeling engine should do something -- instead an action represents *what* the modeler intends for the agents to do. (Technically, we might say that Actions takes a "declarative" approach instead of an 'imperative' approach, but that's not quite true, because actions do allow us to define behavior in a richer way than most declarative approaches, and many action constructs map directly to imperative approaches.)

Actions are connected together in a series of sources and targets. (Technically, an acyclic directed graph.) In an abstract sense that is similar to the way any programming language is defined although that structure isn't usually obvious because of the constraints of textual representations. But unlike a typical programming language, in Actions it is not the execution thread (the processor) that moves from one instruction to the next, but the result of the previous action. In this way, action results "flow" from the output of one action into the next action.

Why are these distinctions between traditional Object-Oriented and the Action approaches important? They give us the advantages of simplicity, clarity and flexibility that data-flow approaches like spreadsheets and some query languages have, but with less restrictions. At the same time, they can bring us much of the power and expressivity of functional languages like Lisp or logical languages like Prolog, but without the level of complexity and obscurity that such languages can have.

We can get a better idea for how Actions work by thinking about how a spreadsheet works. In a spreadsheet, we might define a cell A that adds up a row of data, say "Income". We might define another cell C ("Profit") that takes A and adds it to another cell B that adds up another row of data ("Expenses"). Now, if we change a value in any of the rows, all of the other rows are added up and we get the results in A and B updated automatically. We never had to write code that said something like "for each cell in row X, where...". In fact, we don't really care *how* our Spreadsheet program adds up the numbers -- it could have added them all up at once but in backward order, or stored a running total somewhere and updated just the difference in value for the cell we changed -- what we care about is *what* the result is, and whether it is correct.

But Actions are much more powerful than a spreadsheet, because what is flowing from Action A to Action B is not just a number, but any model component such as a space or a set of agents that we need to use in target actions.

3.2. Design

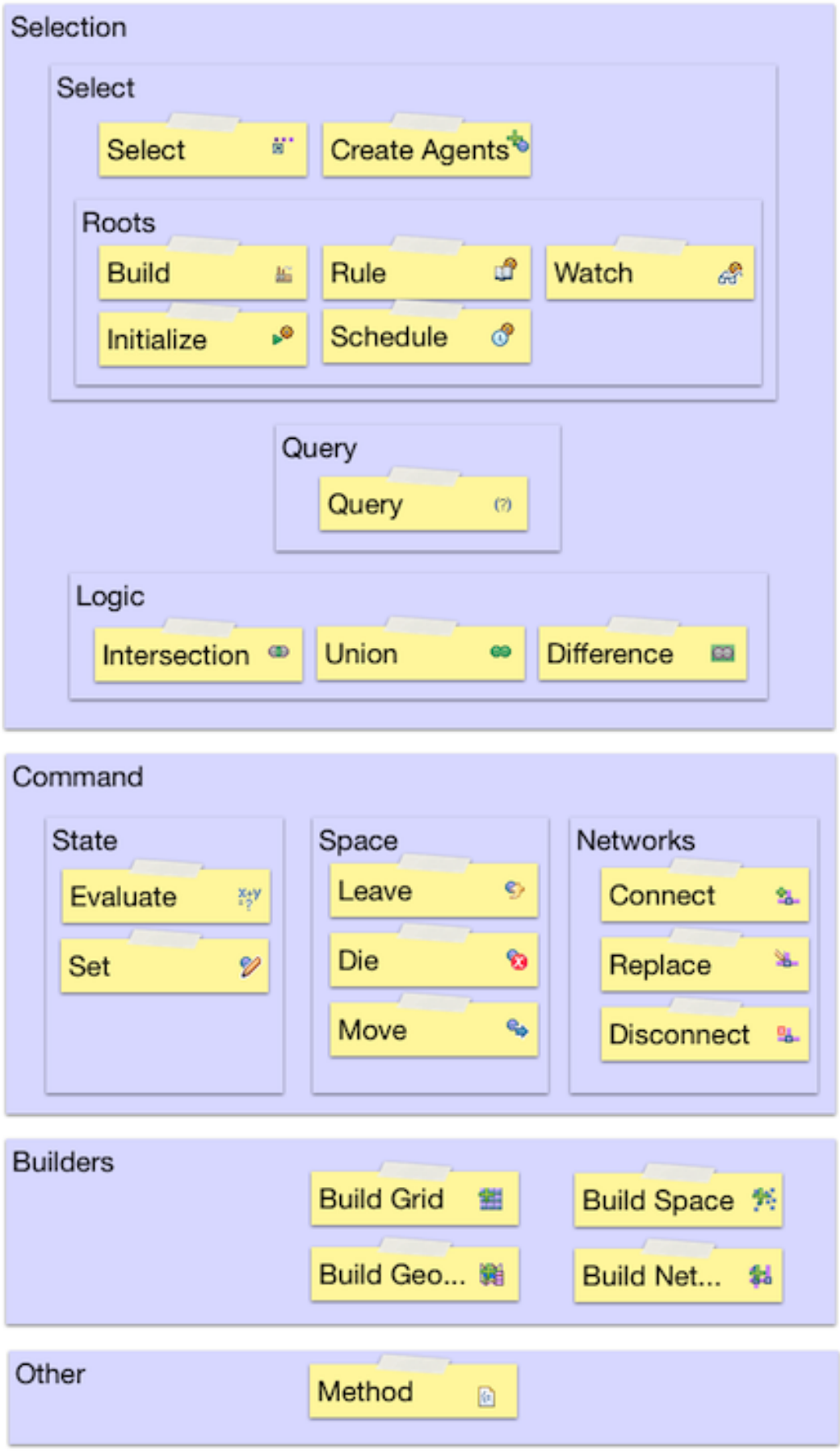
3.2.1. Types

Before getting into the details of how each Actions work together, or the various kinds of Actions, we should understand how they all fit together. As discussed above, actions are strung together in a sequence or flow. They're always composed of two parts, though those parts can be assembled and intermixed in many different ways. First, we search for a collection of agents, and then we do something with that selection. We refer to these two parts as Selections and Commands. There are two special kinds of

selections. First, we need some way to start the whole chain of actions off, and so we have a kind of Selection called a Root Selection, or simply a Root. Secondly, we need some way to actually make the agents exist in the model in the first place, so we have a Create Agents action. For the technically minded, another useful way of looking at the Actions approach is as a Query Transformation language, as with SQL and Stored Procedures. Except again, the results of the queries along with the transformations flow through from one query to the next.

There are two other special categories of actions. One we use to crate the spaces that agents exist upon. We call these Builders. The other category contains just one member, "Method".

These are all summarized in the diagram below.


Selection
Select
Select
Create Agents
Roots
Build
Rule
Watch
Initialize
Schedule
Query
Query
Logic
Intersection
Union
Difference
Command
State
Evaluate
Set
Space
Leave
Die
Move
Networks
Connect
Replace
Disconnect
Builders
Build Grid
Build Space
Build Geo...
Build Net...
Other
Method

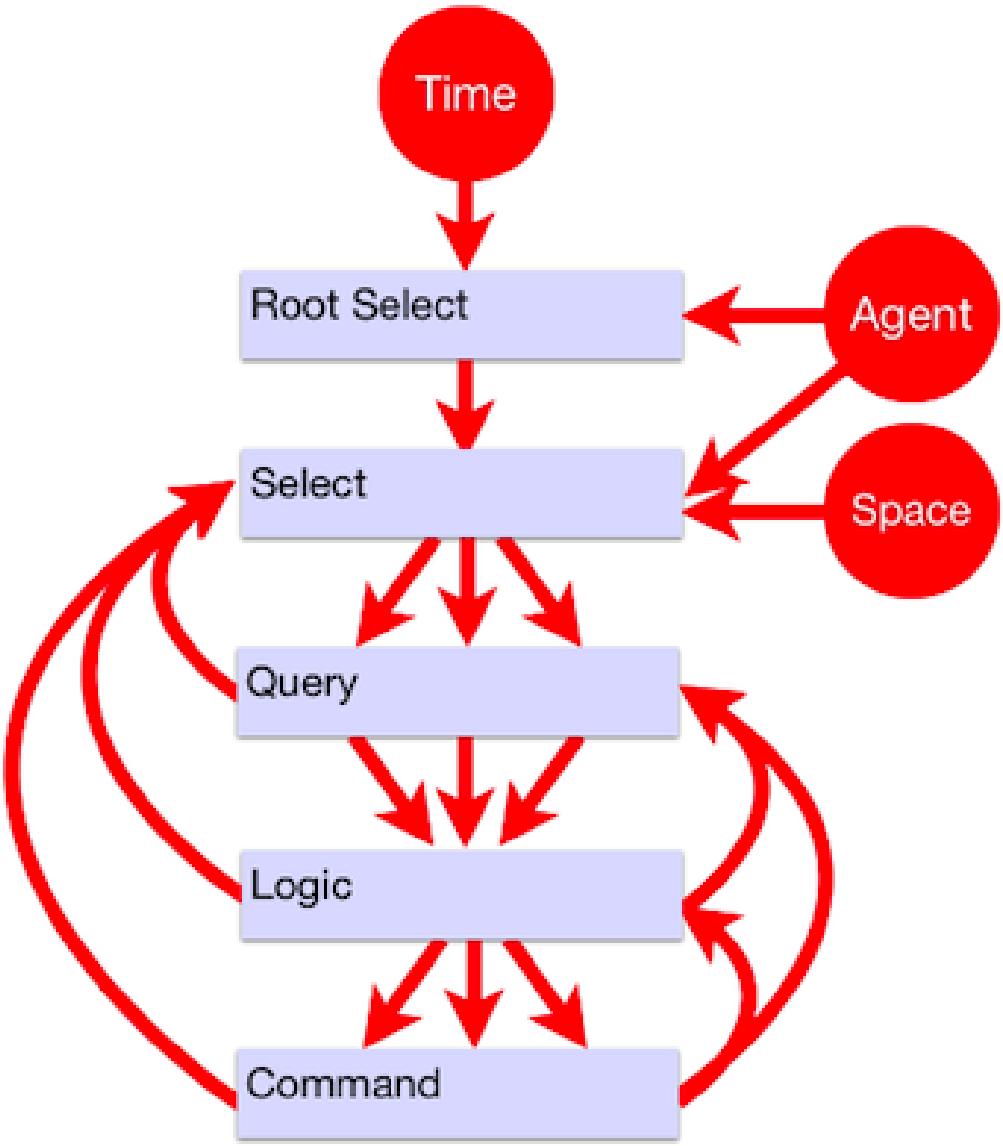
In this section, we'll describe how modelers can assemble actions into sets of behavior that accomplish complex tasks on interelated agents and spaces over time.

3.2.2. Flow

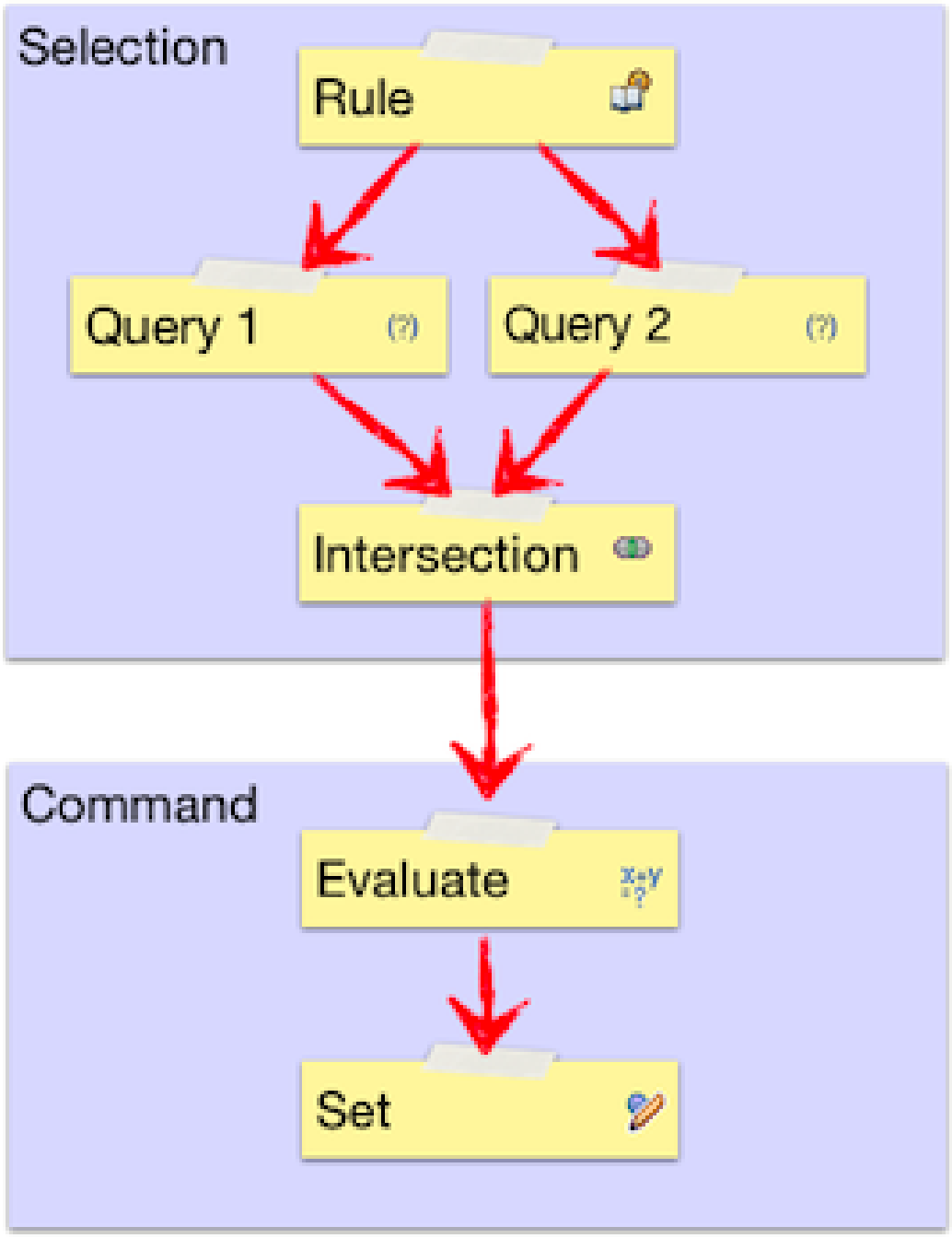
First, let's look at how actions define the basic path that agents take during a model run. As with any programming language, the path we take through the program specification is what determines our state when we get there. In a pure object oriented program, the path just defines the control flow -- what we are doing. The actual state of our model is defined within the object itself. If we call a method B from another method A, we'll be relying on method A to set the values that we need into the object state itself. In a purely functional program the path defines how we are going to deal with whatever has been explicitly passed in to a function that has been called, that is the function parameters. In fact, most languages such as Java combine aspects of both approaches.

In Actions, the path itself implicitly carries all of the context of prior execution with it. This means that we don't have to worry about storing context in the object -- as we would in an object-oriented language -- or passing the correct values from one method call to the next as we would in a functional language. Instead, Actions can use the implicit context of the path of flow to determine what the current state of execution is.

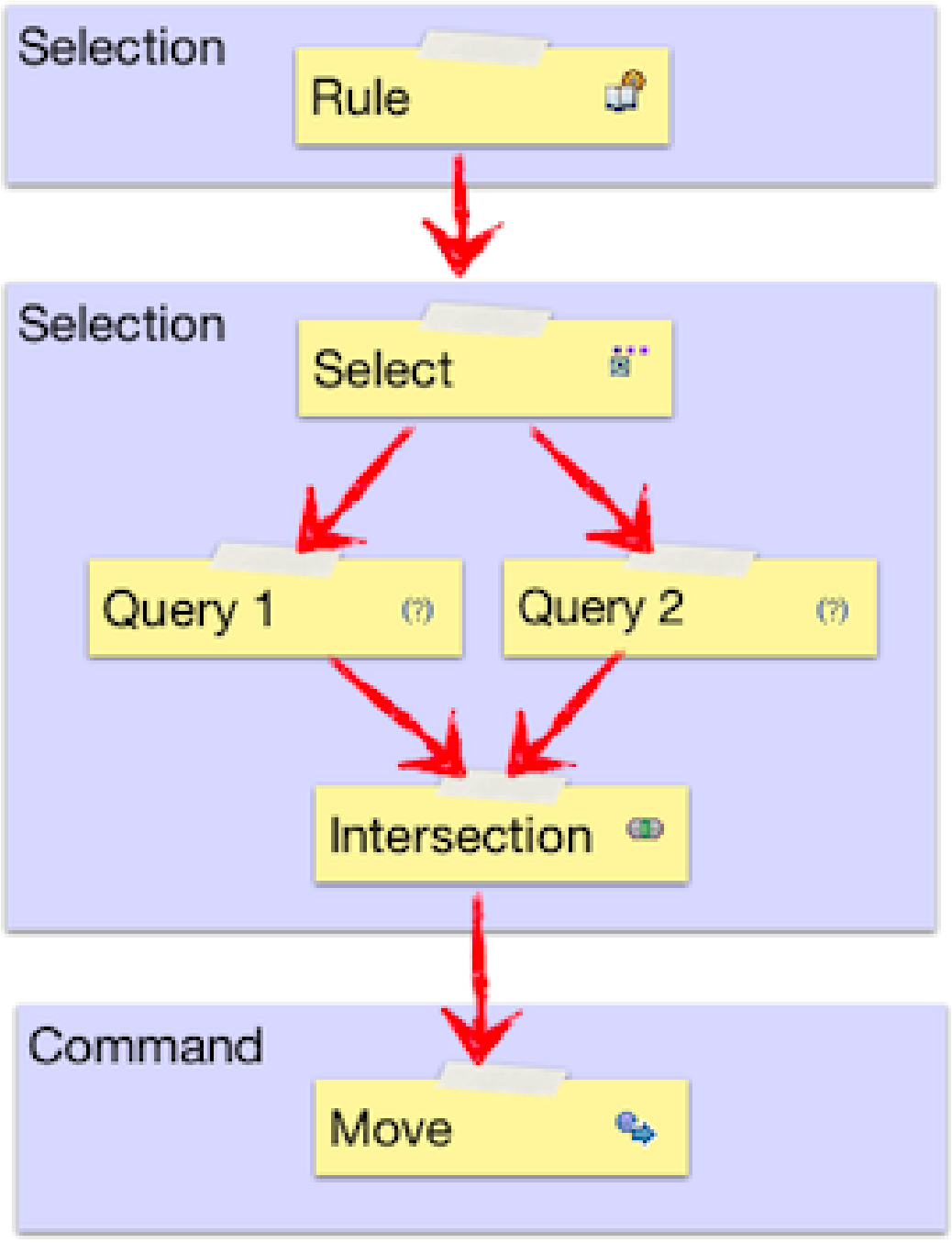
Again, behaviors in Actions are always defined by a set of *selections* and *queries*. In the following diagram, we can see the pattern.



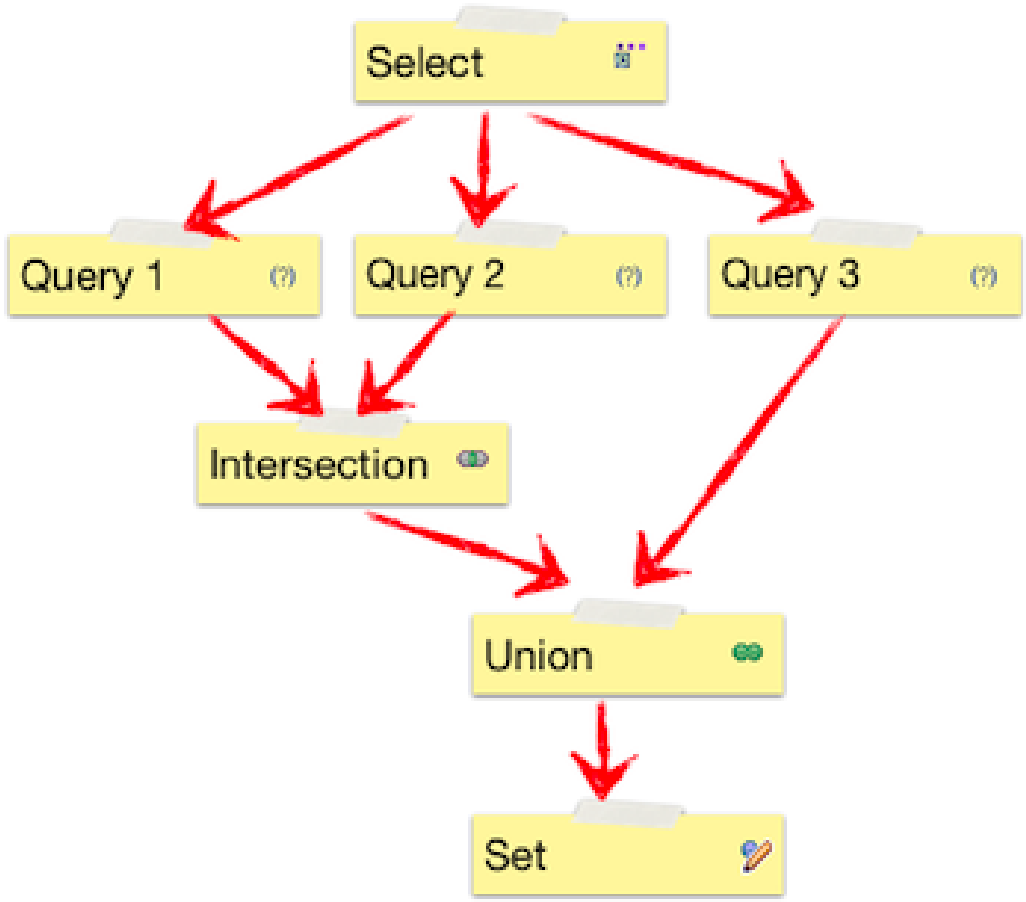
First, we define a Root Selection for a Rule, Schedule or other triggering event. Then, we might add a series of Query and Logic Actions to define the specific agents that we are interested in. These are all part of the Selection. Next, we might define a series of Commands to determine what to do with those agents. Or, we could use the result of that selection to immediately define another selection, for example if we are searching for an agent that is near another agent. The next diagram depicts a simple example.



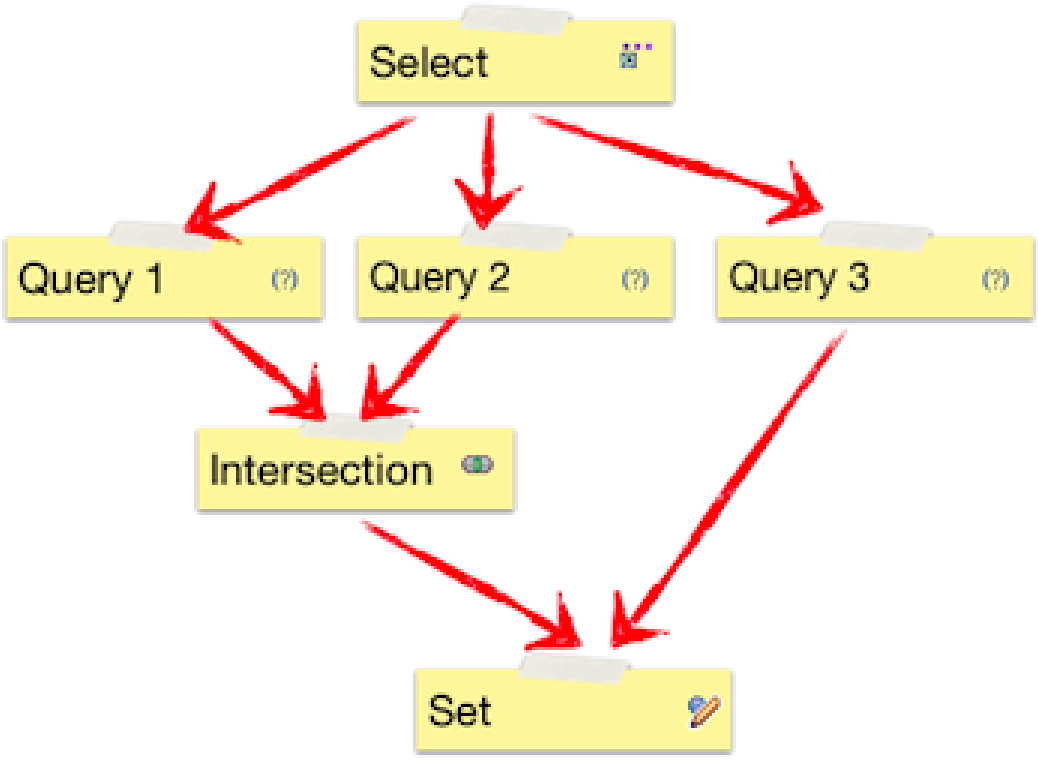
Here, we create a rule, and then check the results of two queries. For any agents that meet those criteria, we'll evaluate some function based on their state, and then set some value on them. In the next example, we'll first create the rule, and then create a new selection with a set of criteria. Finally, we'll do a move based on those queries.



Logic Actions are used to assemble any kind of query structure needed. In the following example, we've defined a selection where the Set action occurs for any agent whoe meets the criteria defined by (Query 1 AND Query 2) OR Query 3. (Readers with experience with programming or formal logic will note that this looks just like a parse tree, and while that's basically what it is, there are important differences from how such trees are generally treated as we'll see later.)

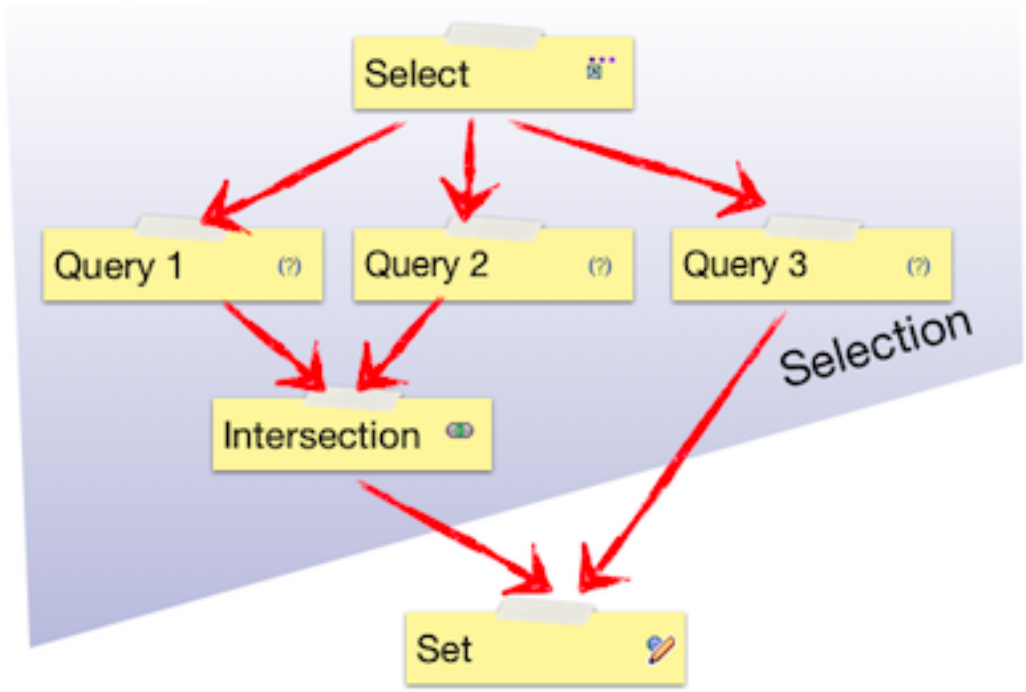


Note that Unions are often superfluous, though they can be useful for organizing queries. This is because all sources that flow into a given target Action act just like a union since any of the paths might reach that target. The following Action design is fully equivalent to the one above.

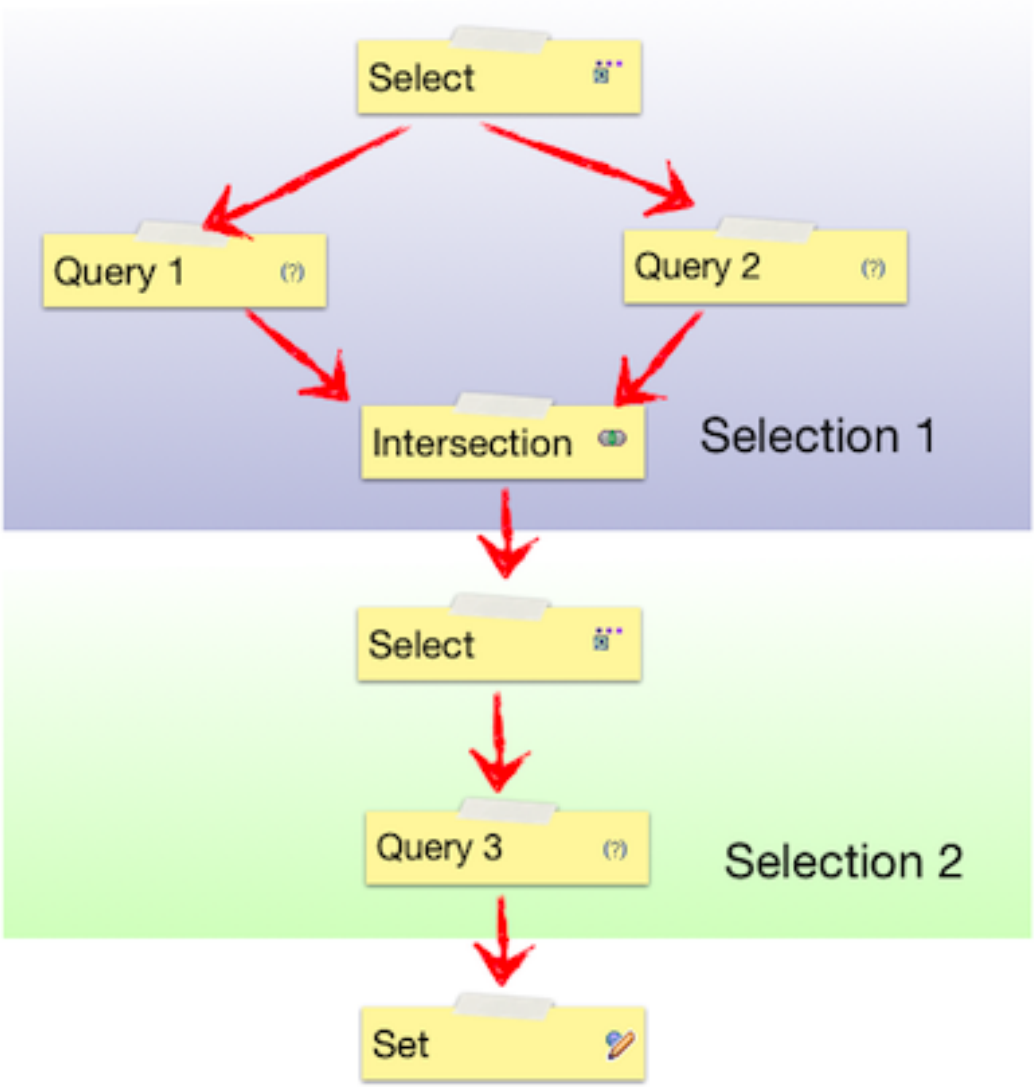


3.2.3. Boundaries and Query Refinement

An important concept in Actions is the idea of selection boundaries. A selection boundary is defined as the limit of a query directly upon a given Select Action. For example, in the following diagram, we can see the extent of the boundary for a straightforward selection.



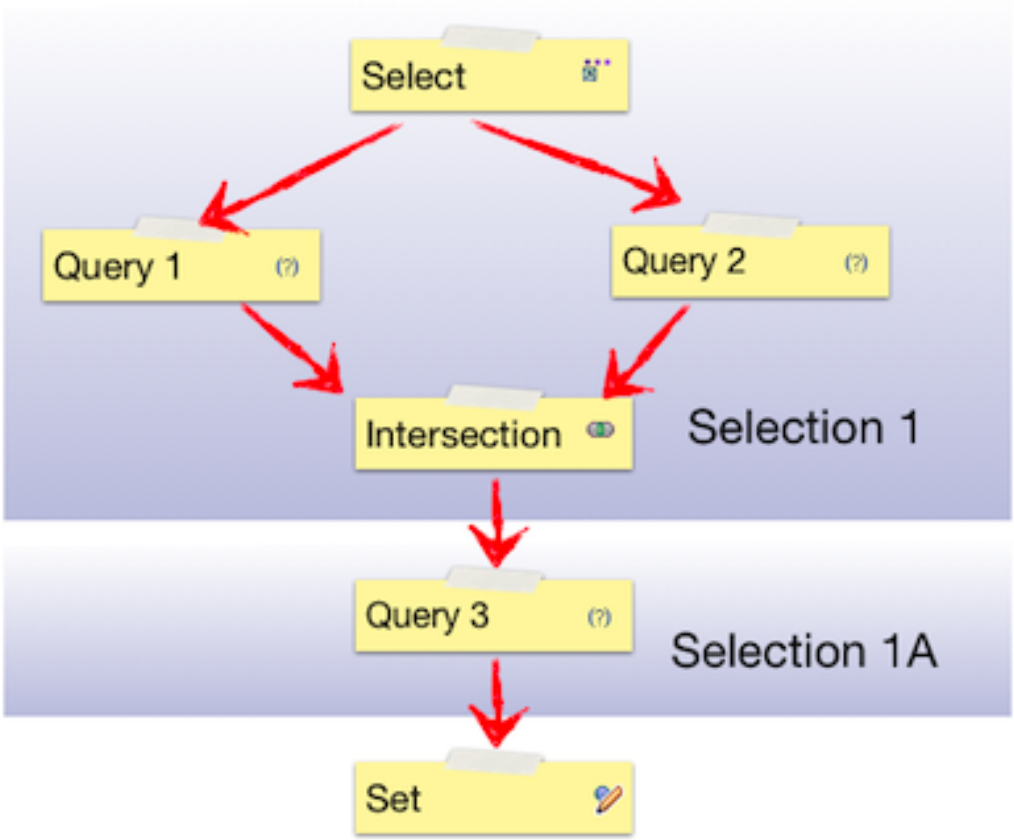
Selection boundaries are defined by Select, Query and Logic Actions. And we each time we create a new selection, we define a new set of boundaries.



A very important aspect of boundaries is that they can be defined by the beginning of Query actions as well as by Select Actions. This is because Query Actions can be directly part of a selection definition, but they can also refine selections, as we'll see. So where does one selection boundary end and the next one begin? The simple rule is that the end of the boundary is defined for a given Action by the place where:

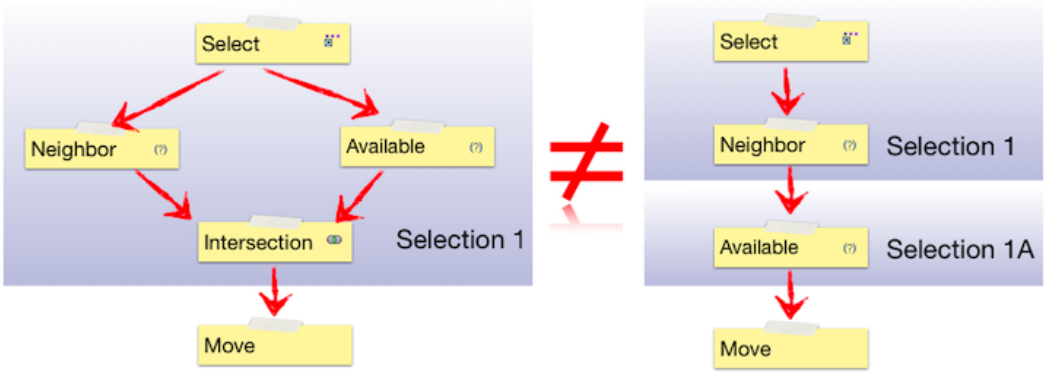
1. A Query Action is not followed by a Logic Action, or
2. A Logic Action is not followed by another Logic Action

In other words, as soon as a Logic Action occurs in a path leading to an Action, any following Query will define a new boundary, as we can see in the following example.



Note that we refer to "Selection 1" and Selection 1A". This is because Selection 1A is a refinement of Selection 1 along its particular path of flow. When a query appears for the same selection but past a particular boundary, you can think of it as a sort of filter on the selection contents. We don't have a "Selection 2" here because any Actions that refer to "Selection 1" along the current path of flow will be acting on the selection defined by Selection 1 and Selection 1A.

These rules allow actions to be defined in the simplest possible way, but it is important to understand their implication as they result in behavior that can be different from what someone used to and imperative programming environment such as Java might expect. In a simple case the distinction might not matter. For example, if we are using a Query 1 to test whether an agent's attribute a = x and attribute b = y, we would get the same outcome if we placed intersected the queries as if we simply put them in sequence. Internally we would actually be searching for agents with a = x, and *then* taking those agents and choosing those agents with b=y, but the outcome would be the same. But consider a more sophisticated case, where we are searching for neighboring available cells.



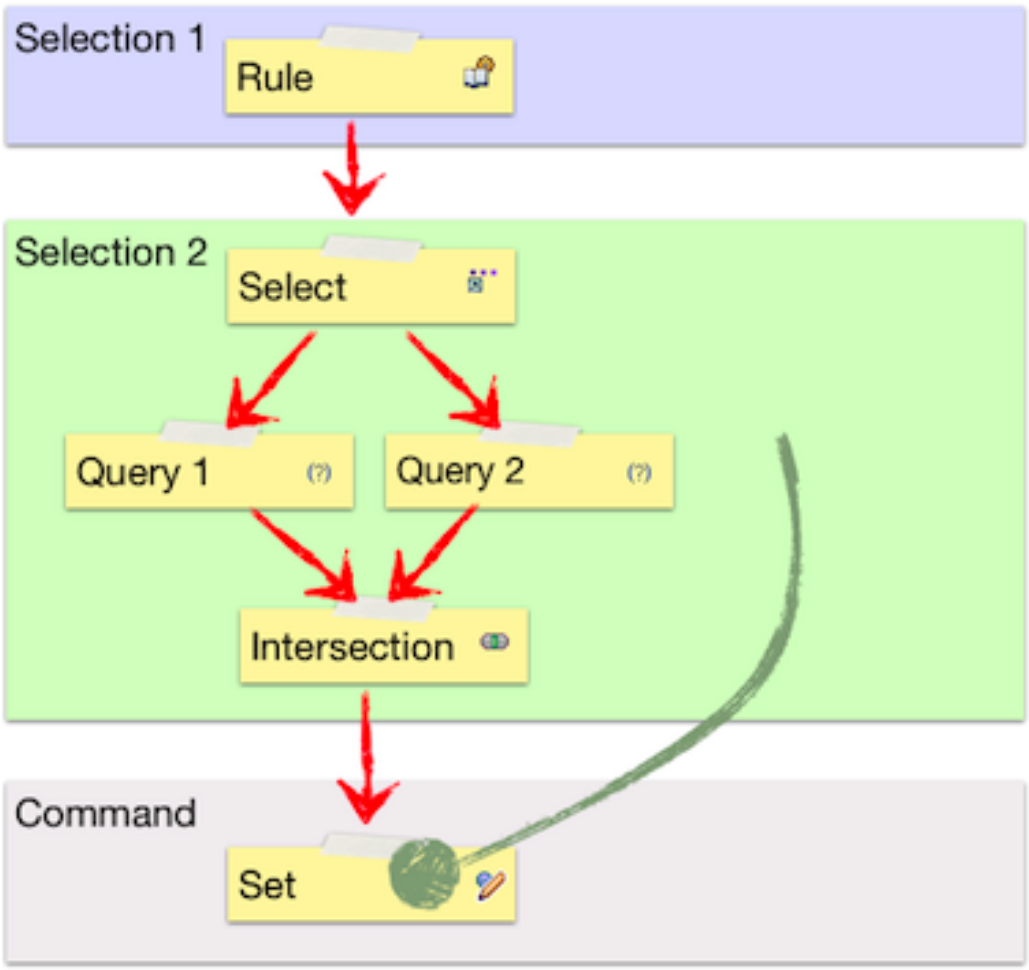
1. "find a cell that is a neighbor and available"

1. "find a cell that is a neighbor"
1A. "is that cell available?"

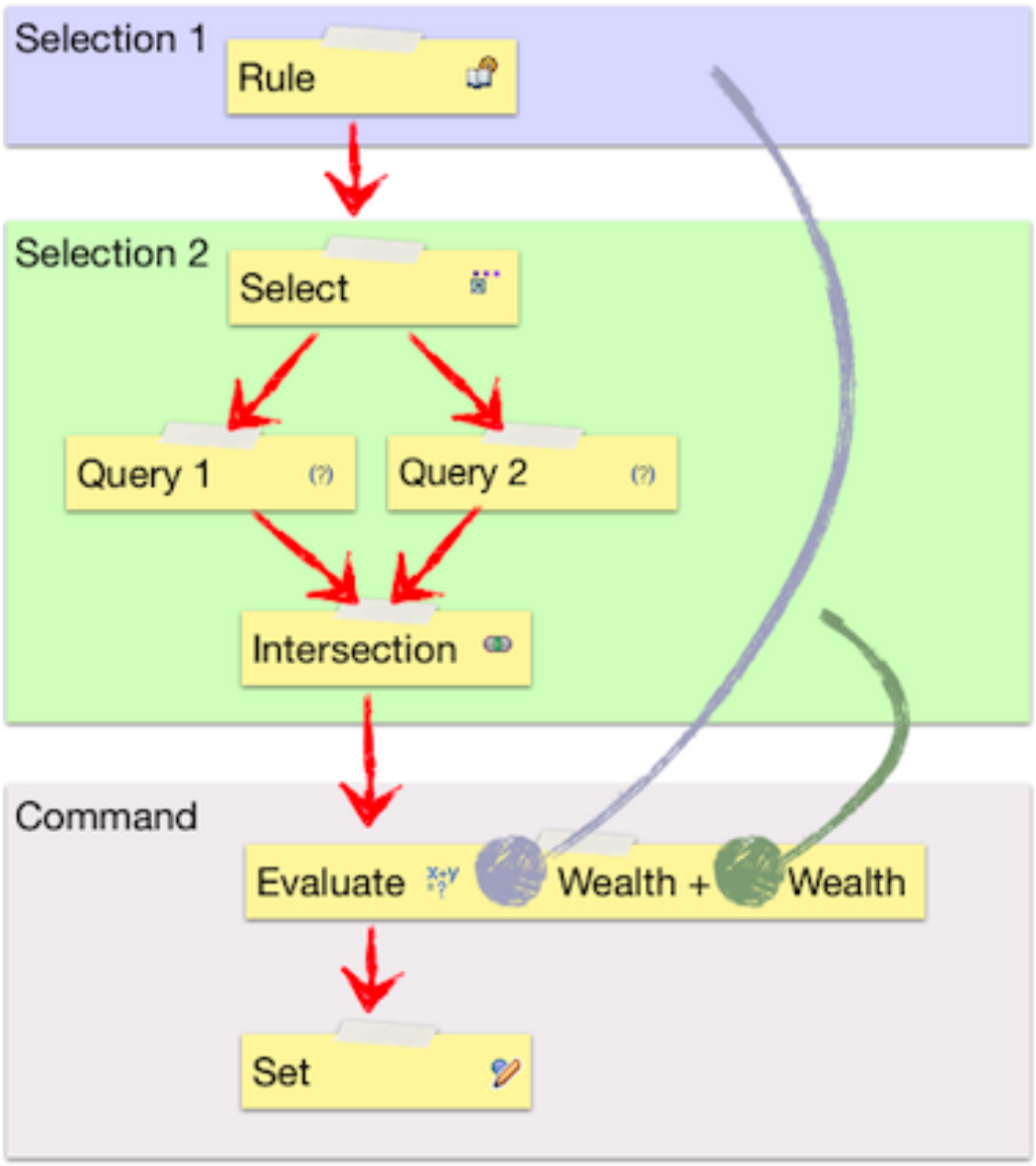
In the first case, we execute a search for all agents that meet the two criteria. This means that if there are *any* neighboring cells which are available, we're guaranteed to find one (random) cell. In the second case, we first search for all cells that are neighbors. This will match any agents that include both available and non available agents. Now, at this point since our search returns one agent (in the current AMF design -- richer behavior will be supported in the future) the randomly selected agent could be either available or not. So in the second case, we might end up with no cell to move to, and thus make no move at all. This then becomes an important aspect of model design. For example, if one were defining a model where neighbors played a game with each other, one might want to instruct agents to play the game only with neighbors that have a certain wealth threshold. In the real-world situation that we are modeling, we might simply search for neighbors who are over a given wealth threshold and then play the game with them. This would imply that information about other agent's wealth is open knowledge. Or, we might simply select a random neighbor, and ask that neighbor to play a game with us. Upon discovering that our neighbor does not meet our wealth criteria, we would then choose not to play with them. Here we are modeling a cost in time to obtain information about another agent's wealth, because we might miss an opportunity to play the game with another agent on that round.

3.2.4. Weaving

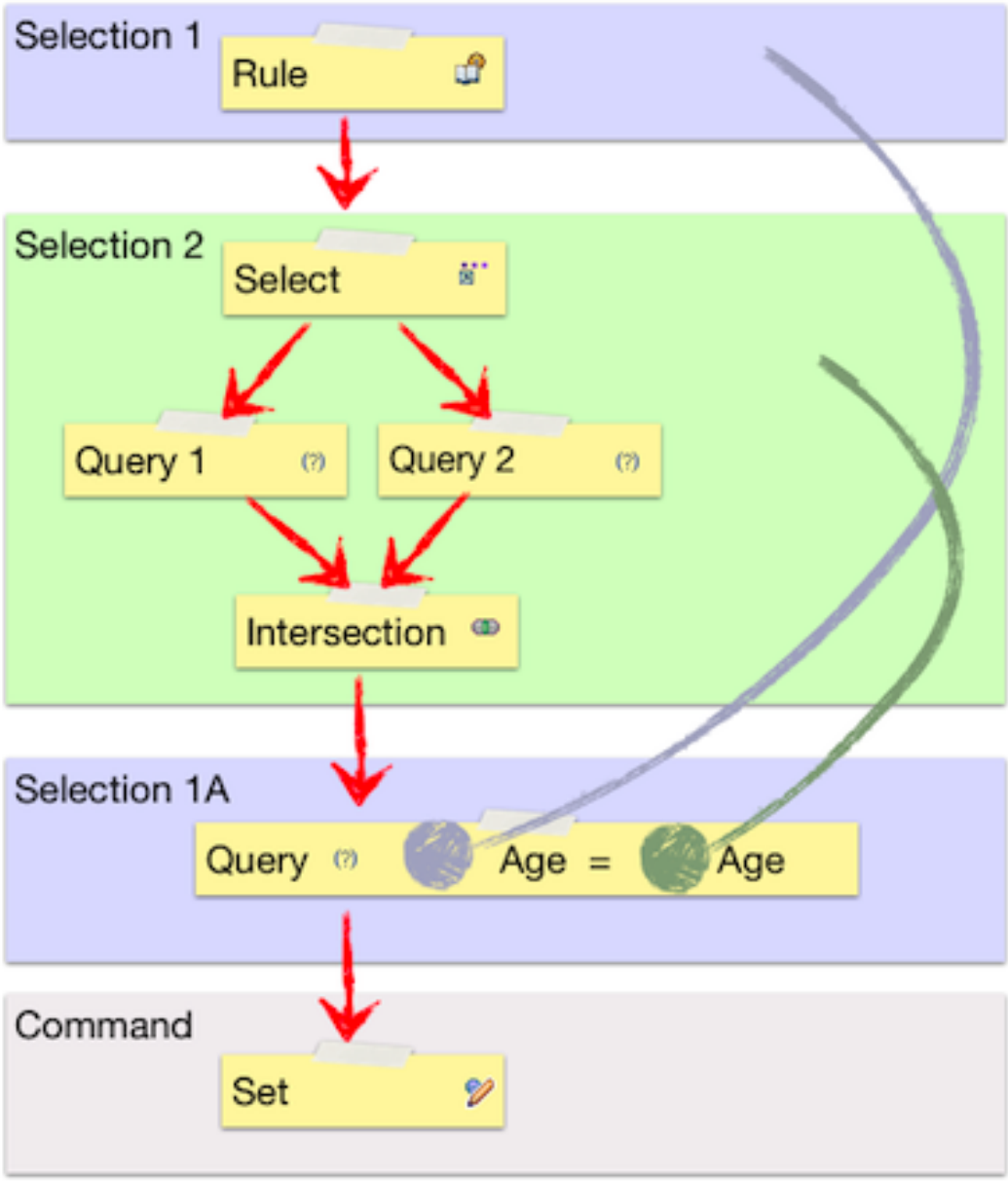
Now, let's put the concepts of Actions sequences and boundaries together to see how we can easily define complex interactions between multiple selections. When we define a Select, the state of its selection flows through and wtih any subsequent selections. So for example, if we have a Root Action rule, and then do a selection based on it, we'll have access to the agent from the orginal context as well as all of the subsequent selections. We can refer to any previous selection for any subsequent action. For example, instead of setting the value for the rule agent, we might instead set a value for an agent we've found in a target selection.



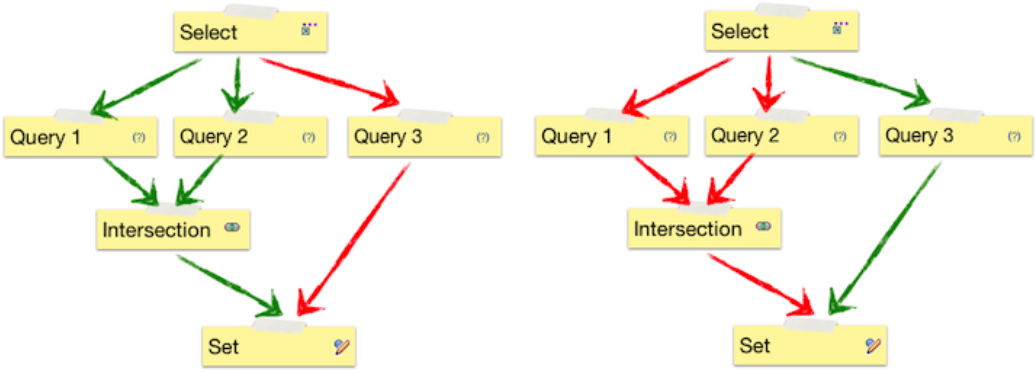
Inputs to functions also use selections. (We'll discuss more details in the functions section.) In the following example, we're adding the weath of the Selection 1 agent to the wealth of the Selection 2 agent and using that value to set some other value. (Here, perhaps we are modeling an agent in a winner takes all game, in which case we'd also add a Set Action on Selection 2 and set the second agent's wealth to 0.)



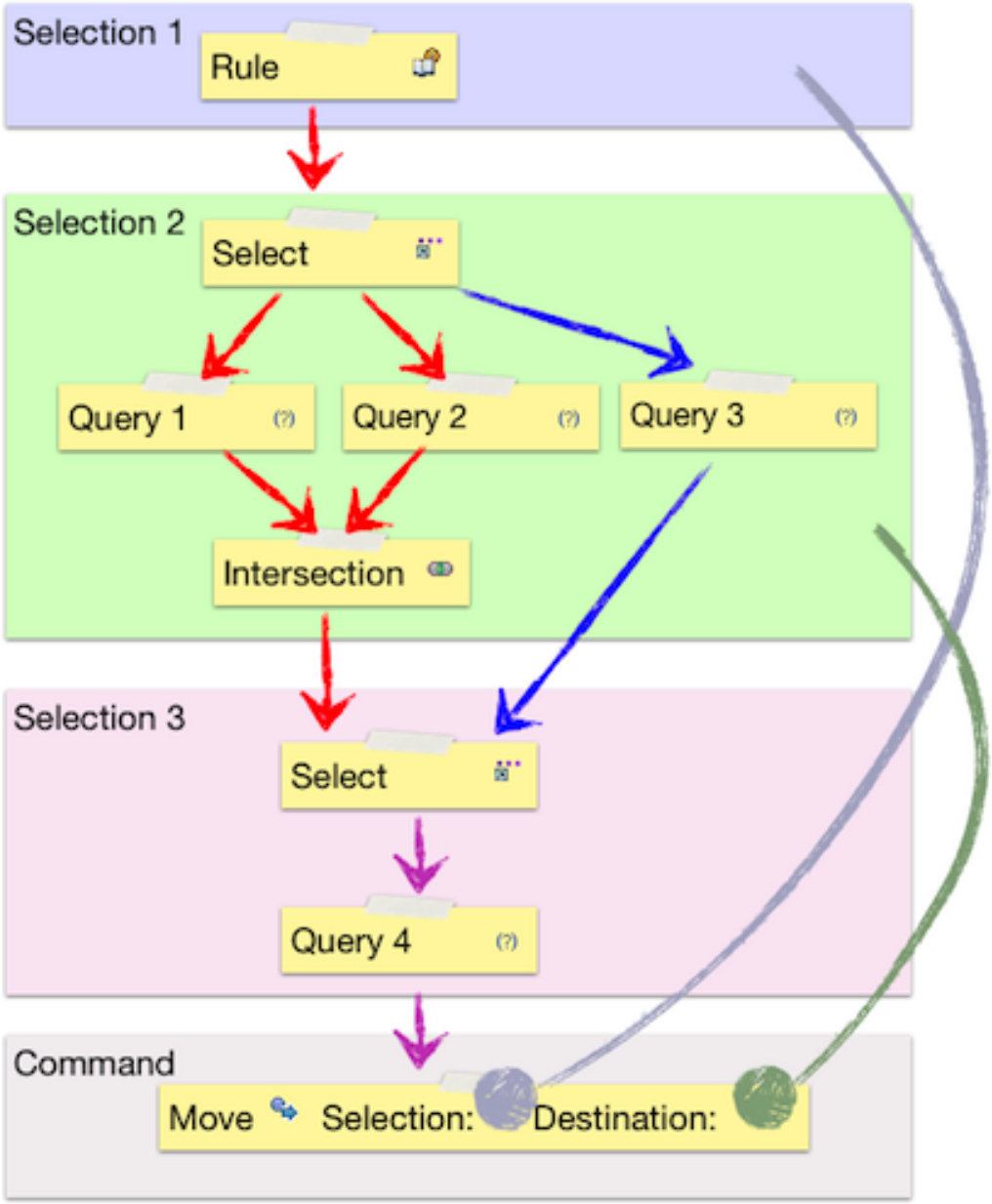
But we can also use selections in defining Query Actions themselves. So in the following example, we select a neighbor agent and then compare the age of our Rule agent with the age of the Selection 2 agent. If and only if those ages are the same will we execute the target Set Action. This example also demonstrates why we refer to the data flow as weaving. Query Actions can be used to refine selections at any point in the data flow. Selections and their uses are interwoven throughout an action sequence.



Now, we can combine this weaving with the sequence flow described above. In the simplest case, this again behaves just like any other program control flow. In the example below, as you have probably already assumed, the agents that have the Set Action applied to them could take one of two paths through the Action flow, either by meeting the criteria for Query 1 AND Query 2, or the criteria for Query 3.



If we use multiple paths in the Query, the path we take though any prior Query will determine which agents are used as the basis for subsequent selections. So in this case, the Rule agent might actually move twice if it meets the criteria for both the blue path and the red path.



In the above example, we introduce one more important concept. As we'll see in the detailed explanation of each Action below, Transformation Actions such as Move or Connect take multiple selections. The first selection defines the set of agents that will be performing the action. In the case of a Move agent, this refers to the mover. The second selection, which for Move we call "destination", refers to the selection that will be recieving the action. In the case of movement this is the agent or location that the Rule agent will be moving to.

3.3. Components

A selection defines a particular set of agents that we want to do something with. Selections are made up of the Select action itself, along with Query and Logic actions. When we refer to a selection in any target command, we are referring to the selection in the context of where we have defined the behavior. We'll see how that works, below.

3.3.1. Root Actions

Root actions are a special case of a selection. These represent behaviors that are defined for all members of agents; these are the highest granularity of an agent's behavior, such as "Find Partners" or "Eat" or "Reproduce". When you want to create a new set of Actions, you have the following choices.

3.3.1.1. Build

The Build Action is a specialized action that allow the construction of member agents and spaces within a parent context. A Build Action executes once for each context before any initialization actions occur for any children agents of that context. Currently it is undefined wether a context's own Initialize Action is executed before the Build Action occurs, so implementors should not rely on any initialized values being available at build time.

3.3.1.2. Initialize

An Initialize action is excuted once and only once for every agent when the model is first started -- at time 0. Initialize Actions are guranteed to execute before any other non-builder action occurs.

3.3.1.3. Rule

A Rule executes once for every agent for every iteration of the model. An important note is that the actual sequence of rules is technically undefined. An implementor should not rely on the fact that a rule occurs before another rule in the list of agent actions though typically the order in which the rules were actually created is respected.

3.3.1.4. Schedule

A schedule is executed on a recurring basis, according to the values detailed below. Note that schedules are often overused. In most agent-based models it makes sense to have any behaviors occur at the same granularity using a Rule. Please note that Schedules are not currently supported in the Escap target, but that support should be available soon. In the following descriptions we refer to period as the current iteration of the model, that is where time = t.

Start

The period that the schedule will first be invoked. For example, if this value is 100, and interval is 1, the schedue will be executed at times 100,101,102..

Interval

How often the schedule is invoked. For example, if this value is 3 and start is 1, the schedule will be executed at times 1,4,7..

Priority

Where the rule will be placed in the execution queue for a given time period. For example, if Schedule A's priority is set to 3 and Schedule B's is set to 2, Schedule B will be executed for all agents before Schedule B is executed for any agents.

Pick

Controls how many agents to execute the schedule against. While this value will have an effect on the Repast target, it is not recommended to be used for general models and is likely to be repalced by another approach.

3.3.1.5. Watch

A Watch action is executed anytime the watched value is set for any agent. Note that the Action will be triggerred even if the state is simply set back to the value that it already has. It is important to be careful about creating Watches that might set the values of other Watch actions which might in turn trigger this watch. To clarify, if a modeler creates a Watch A for attribute a, and creates a target Set for it for attribute b, and Watch B is watching attribute b, then if Watch B has a target Set for attribute A, a circular execution

could occur. This would cause the model to get stuck in its current iteration. To help model developers avoid this case, a warning will be provided if such a set of circular watch dependencies is created.

Attribute

The attribute that will be monitored for change.

3.3.2. Selection

A selection defines what we are searching for (Agent), where we are searching for it (Space), and from whose perspective we are doing it (Selection):

Selection

The selection that we are searching "from". This seems to be the concept that new users have the most difficulty with, but it is key to understanding how Actions works. Just as with any other Action -- a Move command for example -- a selection needs to know what set of agents that it is working with. For example, if we want to define a selection B that finds the cells neighboring a rule B's agent, we would set selection B's selection to A.

Agent

Here we define what agent we will be looking for. In order for this to make sense, the agent has to be related in some meaningful way to the agent whose behavior we are defining. For example, it might be a partner agent or a location that we might want to move to. An agent must be specified unless we are searching within a continuous space, in which case this value should be empty, and the result of the selection will represent some location in that space. In the current version of the framework, we treat destination cell locations as agents, and require that location to be specified as a target agent, but in a future version we'll allow searches without defining an agent within grid spaces as well.

Space

The space that we want to search within. Of course, this must be specified if we use any spatial query terms (see below), but if we simply want to search across all agents it should not be specified.

Represent Queries (Controls) on agents and Transformations, or Commands, on the result of those queries. Queries, Transformations and other in which each child target carries the execution and data context for it's particular path.

For

This value is obsolete and will be replaced with a different mechanism in the next version of the modeling environment.

3.3.3. Query

A Query represents a concrete criteria for our search. The name is a bit confusing because of potential for confusion with a generic query. Queries -- along with their cousin Evaluators -- define a function that is evaluated and that can take Agent attributes, and the results of other Actions as input. Queries are combined with each other and with the logic actions to determine the results of a selection for their direct target actions.

Function

A query function is evaluated to determine the results of a particular selection. Functions can represent very simple search criteria such as "My Age = Your Age", but they can also representSee the functions section for more information on specific functions.

3.3.4. Logic

3.3.4.1. Intersection

An intersection contains only those agents that match all of its source actions. This is essentially equivalent to a logical AND statement and has similarities to an && operator in a java "if" statement.

But it carries important differences too. For example, there is not neccesarily short circuit execution (it's not needed) and much richer criteria can be joined together.

3.3.4.2. Union

A union contains all agents that match any of its source actions. This essentially equivalent to a logical OR statement and shares similarities with the || operator in a java "if" statement.

3.3.4.3. Difference

A difference contains all agents that do not match any of its source actions. This essentailly equivalent to a logical NOT statement, and has similarities to (and important differences from) the Java else statement.

3.4. Example

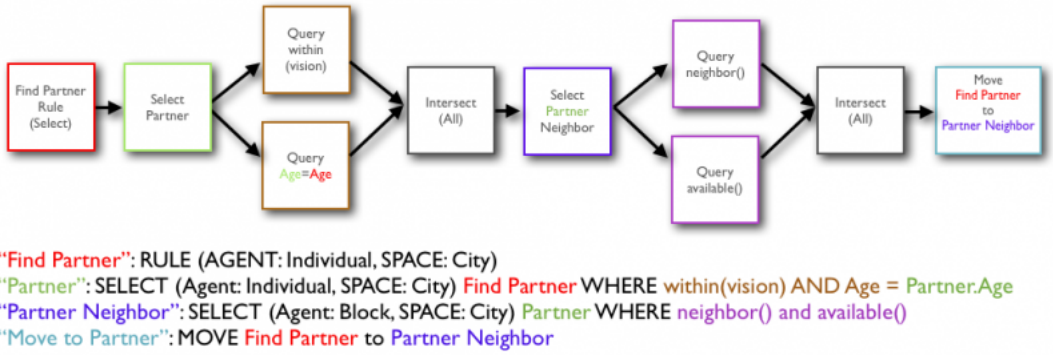
In this section, we'll look at an example that should make clear how the basic Actions approach works in a model. Say we want to define a behavior like:

"Search for a random agent within my vision that is the same age as I am. Find a location next to that agent that is not already occupied

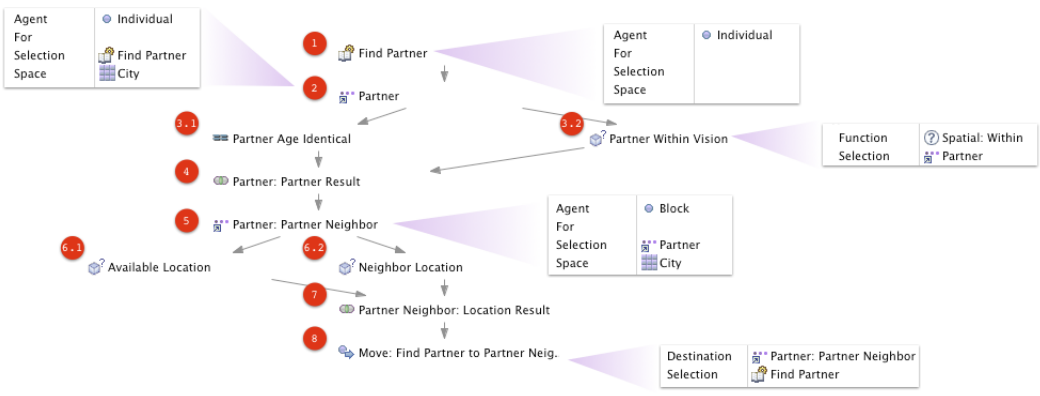
Here, we create a sequence of actions like so:

1. Select every agent for every period of the model. ("Find Partner" Rule)
2. For every member of that selection, search for other agents of the same age within vision distance. ("Partner" selection.)
3. From "Partners" find a random member and search for a neighboring locations. ("Partner Neighbor" selection.)
4. Finally, move the agent in "Find Partner" to the "Partner Neighbor" location.

Now, notice that although it's convenient to speak as if there is only one "Find Partner" and one "Partner Neighbor" in step 4 above, in fact selections are flowing through for each of the results of each of the previous action sequences, and we can refer to each of the directly. We could represent these behaviors in many different ways. For example, we might want to specify the model in a (hand-drawn) graphical language or in a (made-up) textual language:



This is how it looks in an actual model:



And here is how this works in detail:

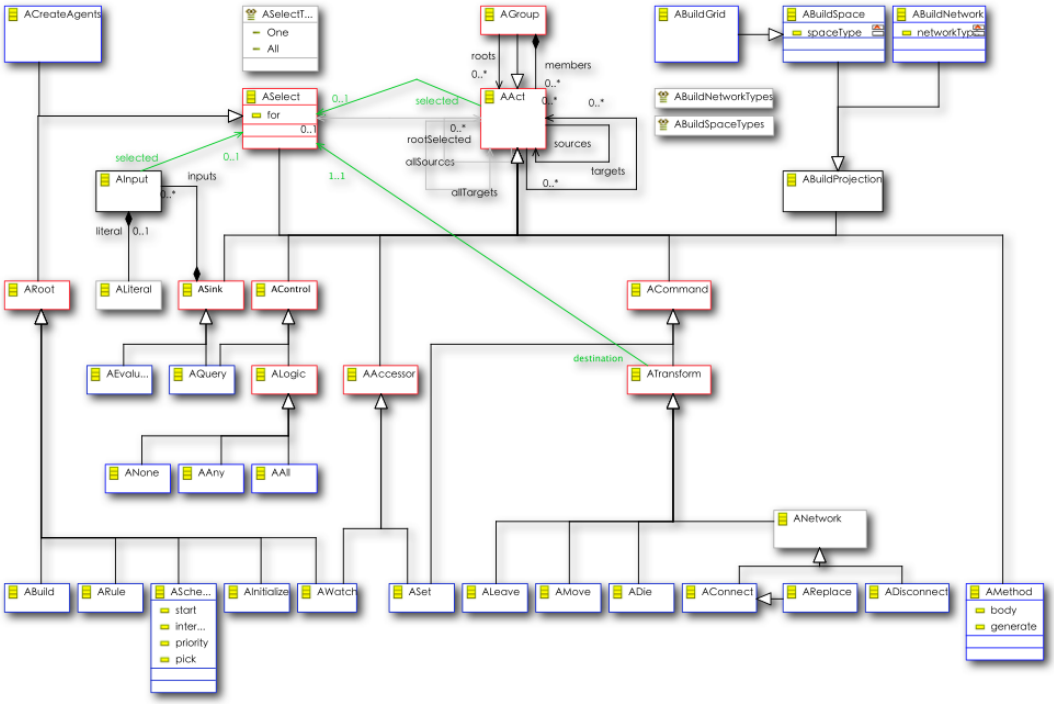
1. Create a Rule that will trigger the behavior. In this case, we want this rule to apply to all "Individual" agents within the model. (Space isn't relevant in this case.
2. Create a child Select Action that will find our partner. Two important things to note here:
   a. The selection occurs based on the "Find Partner" selection. This means that for each Individual in the model, we'll be searching from the point of view of that agent to some other selection of agents.
   b. We also need to define what type of agent we want and in this case the space does matter. We want to find an agent that is nearby within the City space. If instead we wanted to find a partner in a social network, we'd specify that instead.
3. Create two child Query Actions:
   a. We want to search for someone who is the same age as us. This highlights the importance of the idea of the Selection in the Actions design. We're qualifying Age by our
   Partner
   and
   ourselves
   . The selection carries throughout the flow of execution and this context is an explicit part of the entire structure. Note that this is very different from the way control flow works in a traditional imperative language such as Java.
   b. We also want to search using a function for nearness, "within", which takes a parameter of vision. Note that the spatial functions are all polymorphic -- if we decided later on that we would rather search within say "Kevin Bacon space", that is a graph structure representation of space, we would only need to change the space we've defined in Select Partner. (Design discussion note: re-examine this issue WRT possibility of allowing users to search in multiple spaces within the same query.)
4. Intersect the results of these two query components. This delineates the end of the selection definition for any target Actions.
5. Select a neighbor. Again, we can see the importance of Selections. Here, we are selecting from the point of view of the partner, not the initial agent that the current Rule is being executed for. Note that this time our target agent is a "Block", that is, a location within the city.
6. As above, define some queries. This time we want only those agents that are:
   a. available, and
   b. neighbors of our partner.
7. And another intersection..
8. Finally, we move to the location we've found. All that's required at this point is to specify:
   a. The movement selection, or those agents that are moving, which in this case is the original agent we're executing the rule for, and
   b. The destination, which is the cell that we've found. Note that the framework infers from the space definition that the Block agent is capable of hosting the Individual.

3.5. Reference

3.5.1. Diagrams

The following diagram may be helpful to readers familiar with UML and Meta-modeling:

3.5.1.1. Meta-Classes



3.5.1.2. Details

In the diagram above, all meta-objects except for Input, Literal, and the enumerators (lists of options) are Actions. Blue meta-classes are concrete (you can create and use them directly). Red meta-classes are key collaborations.

1. An Act is anything that might happen during the execution of an Agent-Based Model.
2. All Actions have as their root-most source action a Root. These are added first to any agent behavior and act as triggers for all target behavior. For example, a Watch will execute anytime the watched attribute is modified. (As the diagrams do not refer to elements outside of the current package, we cannot see here that Accessor includes a reference to some Attribute, but it does. To see these kinds of relationships you will want to refer to the metaabm.ecore file itself.)#Actions are targets and sources of one another, but an Act can never have itself as a source. (That is, Actions are acyclic, but branches can re-converge. When we refer to an Act source or target, we typically mean to include all ancestors or descendants, not just the immediately connected Act.)
3. All Actions (except for root Actions) reference a Select, referred to as the "selected" relation. An ASelect represents the model aspects that the Act is working within; that is, the spatial, temporal and type (agent) "world" that is currently being selected.
4. Commands trigger some model state change (Set) or spatial transformation (Transform).
5. Controls determine whether target Actions are executed and against what agents. They are in some sense query terms and include Query actions and Logic Actions.
6. Transforms also specify a "destination" Select. This represents aspects that the selected agent(s) will transform to. For example, a Move may use a Rule to select all SugarAgents (type) in the SugarGrid (space) every period (time) and move them to a destination of a neighboring SugarCell (type) in the SugarGrid (space, with time implicit).
7. Sinks are Actions which use some Function (see next section) to interpret state in the form of Inputs. Inputs can come from selected agent attributes, other Actions, or literal values.

Chapter 4. Functions

Functions are relatively simple in terms of model design, but we need to understand how particular functions work in order to develop models. Functions are divided in two ways. By type:

1. Operators are simple calculations sharing the same type.
2. Generics that can represent any general function that takes some well-defined input(s) and returns some well-defined output(s).

And by usage:

1. Functions proper can return any value and are used in Evaluate actions.
2. Logicals return some boolean result and are used by Query actions to decide wether target Actions apply to a particular selection, and by Evaluate actions just as with any other functions. Input types should be defined as generally as possible.

A technical note: conceptually, functions can return multi-values, but that is not currently implemented in the reference targets because of limitations of the target language Java.

These overlap, so we have

4.1. Reference

4.1.1. Diagrams

The following diagram may be helpful to readers familiar with UML and Meta-modeling:

4.1.1.1. Meta-Classes

